ARCHITECTURE DECISION RECORD

ADR-002: Leveraging NATS Core over mTLS for Peer Discovery, Authentication, and WebRTC Signaling

Status:	ACCEPTED	**Date:**	2026-08-23
Deciders:	System Architecture Team	**Scope:**	Signaling Plane, Registry Architecture, Rust Daemon

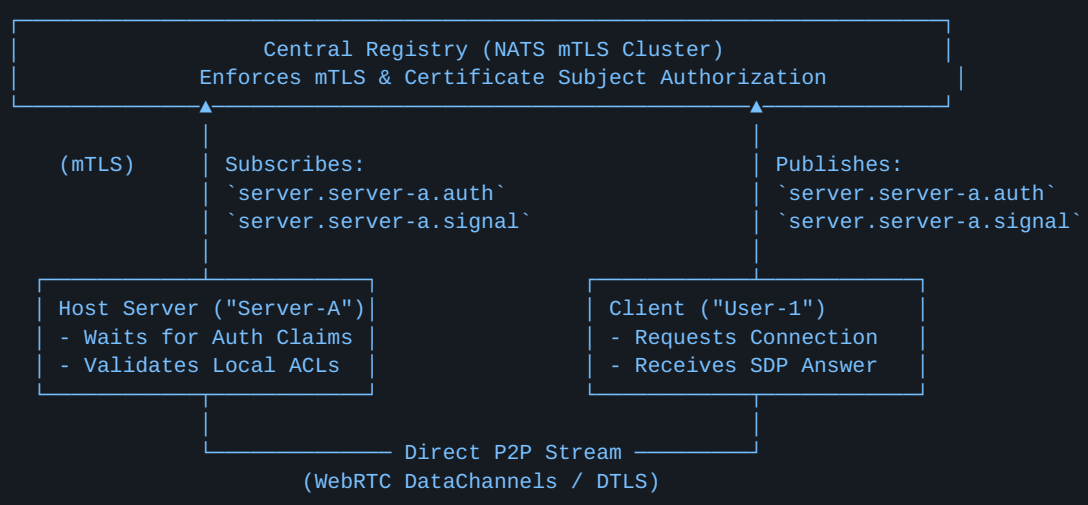
1. Context and Problem Statement

In ADR-001, we established a high-level design for a zero-knowledge, mTLS-secured signaling mechanism to allow WebRTC file transfers between clients and servers. However, building a custom WebSocket signaling relay requires writing significant custom infrastructure code for managing connection pools, subject routing, heartbeat keepalives, backpressure, and horizontal scalability across registry nodes.

We need to evaluate replacing the custom WebSocket registry backend with a **NATS message bus** running over mTLS to act as the messaging substrate for server registration, authentication challenges, and WebRTC SDP/ICE signaling.

2. Decision & Architecture Overview

We will deploy a central **NATS Server cluster** as our primary signaling and discovery backbone. All host servers and clients will connect to NATS over mTLS using the official Rust `async-nats` crate. The registry acts as a NATS message broker with Subject-Based Access Control (Authorization) mapped directly to client X.509 certificate identities.



3. Architectural Features & Justification Analysis

1. Subject-Based Addressing for Server Names

Instead of managing custom in-memory routing tables on a WebSocket server, host servers register by subscribing to isolated NATS subject hierarchies such as `server.<server_id>.auth` and `server.<server_id>.signal`.

Benefits & Trade-offs: NATS handles message delivery, wildcard routing, and endpoint registration natively with zero custom routing code required in our registry backend. This completely decouples client and server network locations, enabling location-blind signaling where clients communicate exclusively through abstract logical channels. The primary trade-off is that server IDs must conform to NATS subject token rules, requiring input sanitization during server naming.

2. Native NATS Request-Reply for Auth & SDP Exchanging

We utilize NATS built-in Request-Reply primitives (which auto-generate temporary `_INBOX` reply subjects) to execute client authentication challenges and WebRTC SDP Offer/Answer exchanges.

Benefits & Trade-offs: This eliminates the need to design stateful session correlation IDs or custom JSON RPC envelope wrappers over raw WebSockets, as the `async-nats` client handles request timeouts and async response pairing out of the box. Host servers process incoming requests concurrently and send replies directly to the client's temporary inbox subject. The trade-off is that long SDP negotiation timeouts must be tuned explicitly on NATS request calls to prevent premature client timeout errors during slow ICE gathering.

3. X.509 Certificate Subject Mapping (NATS Auth)

The NATS server is configured with `verify_and_map = true`, automatically extracting client certificate details (such as Common Name or SANs) during the TLS handshake to enforce publish/subscribe permissions.

Benefits & Trade-offs: Authorization is pushed directly to the network edge, ensuring a compromised or malicious client cannot publish or listen to subjects assigned to other host servers. This reduces our application backend footprint, as security boundary enforcement is handled natively by NATS before traffic reaches application code. A minor trade-off is that modifying client permission schemas requires updating NATS authorization config mappings or integrating a NATS Auth Callout service.

4. Native Keepalives and Auto-Reconnection

The `async-nats` Rust client maintains continuous connection health through light binary protocol heartbeats and built-in exponential backoff reconnection handling.

Benefits & Trade-offs: Host servers running in system trays remain reachable across IP address changes, network switching, or temporary Wi-Fi drops without writing custom reconnect loops or state repair logic. NATS binary keepalives consume less than 100 bytes every 20 seconds, ensuring virtually zero CPU/battery drain on client devices. The trade-off is that brief network blips will cause signaling messages sent during the disconnect window to drop unless JetStream persistence is explicitly enabled.

5. Horizontal Clustering & Zero-Downtime Scaling

Multiple NATS servers can be clustered into a full mesh without introducing central database dependencies or state synchronization layers.

Benefits & Trade-offs: If client demand grows or registry nodes go offline for maintenance, clients and host servers transparently fail over to secondary cluster nodes while maintaining active subject subscriptions across the bus. This delivers enterprise-grade signaling availability without requiring complex custom load-balancing logic. The trade-off is that cluster node inter-communication requires managing cluster mesh ports and TLS certificates across all registry instances.

4. Summary Comparison: Custom WS Relay vs. NATS Bus

Replacing the custom WebSocket signaling engine with NATS dramatically improves developer velocity, operational stability, and system maintainability.

- **Code Reduction:** Eliminates ~1,500+ lines of custom connection management, ping/pong tracking, and thread-safe routing code in the registry.
- **Security Standard:** Aligns fully with mTLS standards, leveraging NATS native certificate subject authorization.
- **Resource Efficiency:** Maintains a negligible idle footprint (< 100 KB RAM) while guaranteeing instant message delivery for signaling events.